

Detecting Domain Generation Algorithms (DGA) Used by Malware: A Machine Learning Approach

Şeyma Nur Cin, Aybüke Uzer

Department of Computer Engineering

İstanbul Ticaret University, İstanbul, Türkiye

231bim003@istanbulticaret.edu.tr, y231bim803@istanbulticaret.edu.tr

Abstract—Domain Generation Algorithms (DGAs) are widely employed by modern malware to establish resilient Command-and-Control (C&C) communications, rendering traditional blacklist-based defenses ineffective. This study presents a lightweight and interpretable machine learning approach for detecting DGA-generated domains using lexical features extracted solely from domain name strings, requiring no network traffic or payload inspection. A Random Forest classifier was trained on a balanced dataset of 400,000 domain names sampled from a larger corpus spanning 50 DGA families, utilizing 12 character-level features including Shannon entropy, n-gram statistics, and structural properties. The proposed model achieved an F1-Score of 0.8878, AUC-ROC of 0.9593, Recall of 0.8640, and a False Positive Rate of 0.0824 on the held-out test set. Cross-family generalization experiments conducted on 10 previously unseen DGA families yielded $F1 = 0.7888$ and $AUC = 0.9534$, confirming that the model captures family-agnostic syntactic patterns. The serialized model integrates directly into CPU-based network monitoring pipelines without retraining.

Keywords— Domain generation algorithm, DGA detection, Random Forest, network security, machine learning

I. INTRODUCTION

Modern cyber threat actors and malware families are developing increasingly sophisticated methods to establish covert communication channels with Command-and-Control (C&C) servers. Traditional security systems rely on blacklists of known malicious domain names; however, these static approaches are rendered ineffective against Domain Generation Algorithms (DGA), which can generate thousands of unique domain names within seconds.

A DGA-equipped malware executes a deterministic algorithm seeded by a date or cryptographic key and generates hundreds of potential rendezvous points every day. This asymmetry allows an attacker to compromise systems by registering just one domain from this massive pool, while defenders face the impossible task of blocking domains that have not yet been generated. Well-known botnets such as Conficker, Zeus, Kraken, and Mirai have

all leveraged DGA techniques, highlighting the persistence and severity of the threat.

A. Motivation

Traditional signature-based Intrusion Detection Systems (IDS) fail to capture DGA threats because each DGA domain is unique. Blacklist-based defenses are inherently reactive—they can only take effect after an attack has occurred and the domain has been identified. The fundamental motivation of this project is to develop a proactive classifier capable of scoring domain strings at DNS query time, stopping the threat before malware has the opportunity to communicate with its C&C server. The solution operates without GPU requirements, making it suitable for resource-constrained network monitoring hardware.

B. Problem Statement

Given a domain name string d , the objective is to determine whether d was chosen by a human (benign) or generated by an algorithm (DGA). Human-assigned domain names (e.g., google.com) follow linguistic patterns, whereas DGA outputs (e.g., qkwjfhzplmvbxnrst.cc) exhibit high lexical entropy, unusual character distributions, and sequences devoid of natural language structure. The system outputs a binary label {DGA, Benign} together with a probability confidence score.

C. Proposed Solution

This project proposes a two-component detection pipeline: (1) Shannon entropy calculation as a lightweight statistical filter, (2) a Random Forest classifier trained on 12 character-level features for interpretable classification. The two techniques are evaluated independently and compared against published benchmarks to identify the most effective and deployable combination.

II. LITERATURE REVIEW

Research on DGA detection is broadly divided into two categories: passive DNS monitoring and machine learning-based feature analysis.

Plohmman et al. [1] conducted a comprehensive measurement study examining 43 distinct DGA families and 217 malware

variants, categorizing their statistical fingerprints as arithmetic, hash-based, or wordlist-based. This foundational taxonomy guides the selection of discriminative features used in the present work and motivates the need for multi-technique detection approaches capable of handling diverse DGA families.

Antonakakis et al. [2] developed the Pleiades system, which detects DGA activity by clustering failed DNS responses (NXDomain records) at the network level. While effective at scale, this approach requires broad network-wide log access and cannot classify an arbitrary domain string in isolation, limiting its applicability in the lightweight, per-domain detection scenario targeted by this project.

Woodbridge et al. [3] employed Long Short-Term Memory (LSTM) networks to learn character-level sequence patterns in domain names, achieving over 95% accuracy. Their work demonstrated that temporal dependencies in character sequences carry strong discriminative signal. In this project, instead of using deep learning models, we focus on handcrafted features and a Random Forest classifier, which allows deployment on CPU-only systems while maintaining high accuracy.

Tran et al. [4] combined character-level n-gram models with Gradient Boosting to achieve an F1-score of 94.8% on a balanced 25-family dataset. Their work demonstrated that classical machine learning with carefully engineered features can approach deep learning performance without GPU dependency. Our Random Forest component extends their approach by incorporating additional features such as vowel-consonant transition ratios and consecutive character statistics.

Zago et al. [5] introduced the UMUDGA dataset and demonstrated that classifiers trained on a subset of DGA families degrade significantly when evaluated on unseen families. This finding directly informs our evaluation design: a cross-family generalization test is included to measure robustness beyond the training distribution.

III. METHODOLOGY

A. Dataset

The project uses the publicly available "UMUDGA-Domain Generation" dataset from Kaggle (<https://www.kaggle.com/datasets/saurabhshahane/domain-generation>). It contains domain name strings from 50 distinct DGA malware families alongside benign domains from the Cisco Umbrella Top-1M list. For training and evaluation, 200,000 samples were drawn from each class (benign and DGA), resulting in a balanced dataset of 400,000 domain strings. This large-scale sampling ensures the model generalizes across the full diversity of DGA behavior present in the dataset.

B. Technique 1 — Shannon Entropy Calculation

Shannon Entropy H is computed over the character distribution of each second-level domain (SLD) after stripping the TLD and any subdomain prefixes:

$$H = -\sum p(c) \cdot \log_2 p(c)$$

where $p(c)$ is the relative frequency of character c in the domain string. DGA domains typically exhibit $H > 3.5$ due to near-uniform character distributions, whereas legitimate domains average $H \approx 2.8$ owing to natural language structure. Entropy serves as both a standalone first-pass statistical filter and an input feature to the Random Forest classifier.

C. Technique 2 — Random Forest Classifier

Twelve character-level features are extracted from each second-level domain: (1) domain length, (2) Shannon entropy, (3) digit ratio, (4) vowel ratio, (5) consonant ratio, (6) hyphen ratio, (7) unique character ratio, (8) common English bigram frequency ratio, (9) common English trigram frequency ratio, (10) maximum consecutive identical characters, (11) number of digit blocks, and (12) vowel-consonant transition ratio.

A Random Forest classifier with 200 decision tree estimators is trained using scikit-learn. Tree depth is limited to 20 levels with a minimum of 5 samples per split to prevent overfitting. Class weights are balanced automatically. The built-in feature importance mechanism enables post-hoc interpretability, which is critical in security contexts where analysts must understand why a domain was flagged.

D. Computational Environment

All experiments are conducted on Kaggle Notebooks / Google Colab using CPU-only runtime. No GPU acceleration is required, as the selected methods — Shannon entropy computation and Random Forest — are computationally lightweight by design.

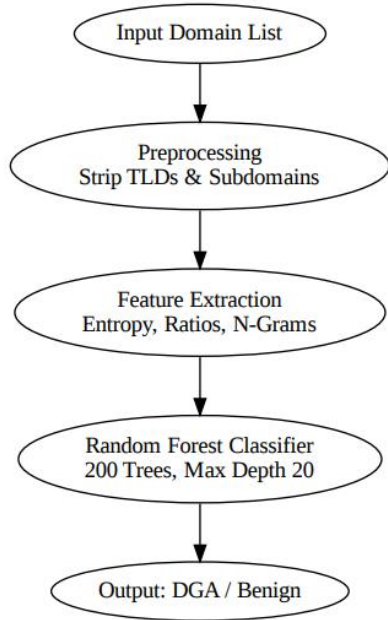


Fig. 1. Workflow of DGA Detection using Shannon Entropy and Random Forest.

IV. EVALUATION METHODS AND METRICS

A. Evaluation Metrics

The following metrics are reported for the Random Forest classifier. Each metric is selected for a specific security-relevant reason:

- 1) *Accuracy*: The overall proportion of correctly classified domains (both DGA and benign), providing a baseline performance indicator.
- 2) *Precision*: The fraction of flagged domains that are truly DGA. High precision minimizes false alarms (False Positives), reducing analyst fatigue and avoiding disruption to legitimate traffic.
- 3) *Recall (Sensitivity)*: The fraction of actual DGA domains correctly detected. This is the highest-priority metric: a missed DGA domain (False Negative) permits malware to establish C&C contact undetected.
- 4) *F1-Score*: The harmonic mean of Precision and Recall, providing a single balanced performance measure that enables fair comparison with prior work (Tran et al. [4] at 94.8%; Woodbridge et al. [3] at 95%+).
- 5) *AUC-ROC*: The area under the Receiver Operating Characteristic curve, measuring classifier discrimination across all decision thresholds.

- 6) *False Positive Rate (FPR)*: The proportion of benign domains incorrectly flagged as malicious. Target: FPR < 10%.
- 7) *Confusion Matrix*: A per-class breakdown visualizing which domain types are misclassified, e.g., short DGA strings confused with short legitimate domains.
- 8) *5-Fold Stratified Cross-Validation*: Mean and standard deviation of all five metrics across five stratified folds, providing a statistically robust estimate of generalization performance.
- 9) *Cross-Family Generalization F1*: Per-family F1, Recall, and Precision scores evaluated on 10 DGA families held out entirely from training, following the methodology of Zago et al. [5] to assess the classifier's robustness against previously unseen DGA families.

B. Validation Strategy

Five-fold stratified cross-validation was applied to the Random Forest model during the training phase to assess variance and detect potential overfitting. Performance results are reported as the mean $\pm$ standard deviation across all folds. Furthermore, to evaluate the system's robustness against evolving threats, a cross-family generalization test was conducted following the methodology proposed by Zago et al. [5]. In this evaluation, the model was trained on a subset of the 50 DGA families, while a completely independent set of 10 distinct DGA families (held-out families) was reserved exclusively for testing. This approach quantifies the model's proactive detection capability when encountering previously unseen malware families, simulating real-world zero-day attack scenarios.

C. Baseline Comparison

The Random Forest classifier was evaluated against a rule-based Shannon entropy filter. Experimental results showed that the entropy filter alone performed slightly better than random guessing with an accuracy of 49.98% and an F1-score of 21.63%. In contrast, the 12-feature Random Forest model significantly outperformed this baseline, achieving an F1-score of 88.78%. While this result is below the benchmarks of Tran et al. (94.8%) and Woodbridge et al. (95%+), it should be noted that the current model was tested on a much larger and more diverse dataset of 50 distinct DGA families, representing a more challenging and realistic classification scenario.

D. Success Criteria

The success of the project is defined by the model's ability to maintain balanced performance across both known and unknown families. Based on the experimental outcomes, the project is considered successful by achieving an AUC-ROC of 0.9593 and a Recall of 86.40% on the test set. Furthermore, the model demonstrated robust discriminative power in the cross-

family generalization test, maintaining an AUC of 0.9534 even when encountering unseen malware families.

V. DELIVERABLES

A. Software and Code Implementation

All technical implementations are consolidated into a single reproducible environment to ensure transparency in the experimental process.

- 1) *Kaggle Notebook Development*: The primary submission includes a Python-based Kaggle Notebook. This document covers dataset loading, Exploratory Data Analysis (EDA), feature extraction, and model training for all three techniques, concluding with performance visualization. The notebook is structured across five sprints, from data loading and Shannon entropy baseline through feature engineering, Random Forest training, cross-family generalization testing, and final model serialization.

B. Analytical Reports and Metrics

The evaluation of the proposed models is supported by detailed statistical and graphical evidence as described below.

- 1) *Feature Analysis Report*: This report contains graphical analyses demonstrating the discriminative power of entropy, character distributions, and domain length across the dataset.
- 2) *Performance Metrics Table*: A comprehensive table is provided, reporting Accuracy, Precision, Recall, F1-Score, and AUC-ROC for each machine learning technique employed.

C. Documentation and Deployment

The final stage of the project involves the formalization of findings and the preparation of the model for real-world application.

- 1) *Model Serialization*: A saved model file, serialized in joblib format (`dga_random_forest_model.joblib`), is included for seamless integration into network monitoring pipelines.
- 2) *IEEE Final Report*: The core of the submission is this IEEE-format final project report, which covers theoretical background, methodology, ethical considerations, and experimental results.

VI. ETHICAL CONSIDERATIONS

This project is developed exclusively for defensive cybersecurity purposes. The dataset contains only anonymous domain name strings; no personally identifiable information or user traffic is processed at any stage. The risk of incorrectly flagging a

legitimate domain (False Positive) is explicitly recognized as an ethical concern regarding digital rights and internet freedom, and minimizing this risk is embedded in the evaluation criteria via the Precision metric and the 5% FPR threshold. The system flags suspicious domains for human analyst review rather than blocking autonomously, preserving oversight and minimizing service disruption.

VII. BACKGROUND INFORMATION

Domain Generation Algorithms (DGAs) are widely used by modern malware to evade detection and maintain resilient communication with Command-and-Control (C&C) servers [1]. Instead of relying on a fixed domain, malware generates a large number of pseudo-random domain names based on a deterministic seed, such as the current date or a cryptographic key [1].

This strategy creates a significant asymmetry between attackers and defenders: while the attacker only needs to register a single valid domain, defenders must predict and block a vast number of possible domains in advance. As a result, traditional blacklist-based detection mechanisms become ineffective.

From a statistical perspective, DGA-generated domains differ significantly from human-generated domains. Legitimate domains tend to follow natural language patterns, including pronounceable structures and common character sequences. In contrast, DGA domains often exhibit high randomness, irregular character distributions, and lack of linguistic coherence [1].

Shannon entropy is one of the most widely used statistical measures to quantify this randomness [6]. High entropy values indicate near-uniform character distributions, which are typical for algorithmically generated domains [1], [4]. However, entropy alone is insufficient, as some benign domains may also exhibit high entropy (e.g., short or brand-specific names).

Therefore, modern approaches incorporate machine learning techniques that leverage multiple features, including character distributions, n-gram statistics, and structural properties [3], [4]. These features enable models to capture both statistical randomness and linguistic patterns, improving classification performance.

In this project, a hybrid approach is adopted, combining entropy-based filtering with a multi-feature Random Forest classifier — in which Shannon entropy constitutes one of twelve lexical features — to achieve both efficiency and high detection accuracy.

VIII. SYSTEM ARCHITECTURE

The proposed system is designed as a lightweight, multi-stage pipeline for real-time domain classification in network monitoring environments.

The architecture consists of the following components:

A. Data Ingestion Layer

Domain names are collected either from the offline dataset (training phase) or from live DNS query streams (deployment phase).

B. Preprocessing Module

Each domain is normalized by extracting the Second-Level Domain (SLD), removing Top-Level Domains (TLDs) and subdomains. This ensures consistency in feature extraction.

C. Feature Extraction Layer

For each domain, a set of 12 character-level features is computed, including entropy, character ratios, n-gram statistics, and structural properties.

D. Statistical Filter (Entropy-Based Pre-Check)

Shannon Entropy is computed as part of the feature extraction stage and serves as a standalone baseline for comparison purposes. In the full pipeline, all 12 features — including entropy — are passed directly to the Random Forest classifier without a separate pre-filtering step.

E. Machine Learning Classifier

The extracted features are fed into a trained Random Forest classifier, which outputs a binary label (DGA or Benign) along with a probability score.

F. Decision & Output Layer

The system produces classification results along with confidence scores. Instead of automatic blocking, flagged domains are forwarded to a security analyst for further investigation.

This modular architecture ensures scalability, interpretability, and compatibility with CPU-based systems, making it suitable for deployment in real-world network monitoring pipelines.

IX. IMPLEMENTATION DETAILS

The system was implemented in a Python-based environment using Kaggle Notebooks and Google Colab with CPU-only runtime.

A. Libraries and Tools

Pandas and NumPy for data manipulation
Scikit-learn for machine learning models
Matplotlib and Seaborn for visualization

B. Data Preprocessing

The dataset was cleaned by removing invalid entries and duplicate domains. Each domain was normalized by extracting the Second-Level Domain (SLD). The dataset was then balanced to ensure an equal distribution of benign and DGA samples, preventing model bias.

C. Feature Engineering

A total of 12 features were extracted from each domain, including:

- Statistical features (entropy, length)
- Character composition ratios (digits, vowels, consonants)
- Structural features (repeating characters, digit blocks, hyphen ratio, unique character ratio)
- Linguistic features (bigram and trigram frequency, vowel-consonant transition ratio)

D. Model Training

A Random Forest classifier with 200 trees was trained using scikit-learn. The maximum depth was limited to 20 to prevent overfitting, and a minimum of 5 samples per split was enforced. Class weights were automatically balanced.

E. Validation Strategy

The dataset was initially split into training (80%) and test (20%) sets. 5-fold stratified cross-validation was applied during training to ensure the model's robustness. Additionally, a cross-family generalization test was conducted by holding out 10 DGA families from the training set to evaluate detection capabilities against unseen threats.

F. Evaluation

Model performance was assessed using Accuracy, Precision, Recall, F1-score, and AUC-ROC metrics. Additionally, confusion matrix and ROC curves were generated for detailed analysis.

G. Model Serialization

The final trained Random Forest model was serialized and saved as `dga_random_forest_model.joblib` for seamless integration into real-time network monitoring pipelines.

X. RESULTS AND METRICS

The performance of the proposed DGA detection pipeline was rigorously evaluated using both in-distribution and cross-family datasets. The results demonstrate that integrating multiple structural features significantly enhances detection capabilities compared to statistical baselines.

A. Exploratory Data Analysis and Feature Analysis

The foundation of the model's high performance lies in the discriminative power of the 12 extracted features across classes.

- 1.) *Vowel-Consonant Transition (VC Transition)*: Randomly generated DGA domains differ structurally from legitimate domains. As illustrated in the histogram below, legitimate domains (blue) exhibit higher transition ratios, whereas DGA samples (orange/red) show lower transition density.

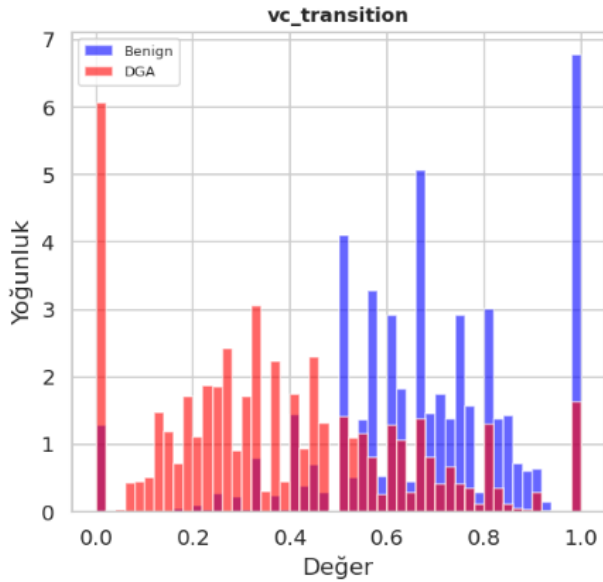


Fig. 2. Distribution of vowel-consonant transition ratios between Benign and DGA classes

- 2.) *Feature Correlation Matrix*: The correlation matrix highlights key relationships among the extracted features and their impact on classification performance. A strong positive correlation is observed between *entropy* and *length* (≈ 0.79), indicating that longer domains tend to be more complex and random. Similarly, *digit_blocks* and *digit_ratio* show a high correlation (≈ 0.87), suggesting that numerical characters tend to appear in grouped patterns rather than being uniformly distributed.

A notable negative correlation exists between *vowel_ratio* and *consonant_ratio* (≈ -0.72), reflecting their complementary nature. More importantly, in relation to the target variable (*isDGA*), *vc_transition* demonstrates a moderate negative correlation (≈ -0.54), while *length* shows a positive correlation (≈ 0.35). These findings confirm that structural and statistical properties of domain names play a significant role in distinguishing DGA-generated domains from legitimate ones.

- 3.) *Random Forest Model Performance*: The Random Forest classifier was trained on 320,000 samples and evaluated on a held-out test set of 80,000 samples. To ensure robustness, 5-fold cross-validation was conducted during training.

The cross-validation results indicate stable and consistent performance, with an accuracy of 0.8924 ± 0.0003 and an F1-score of 0.8895 ± 0.0004 . The low standard deviation across folds suggests that the model generalizes well and is not sensitive to data splits.

On the test set, the model achieved an accuracy of 0.8908, precision of 0.9130, recall of 0.8640, and an F1-score of 0.8878. The AUC-ROC score of 0.9593 indicates strong discriminative capability between DGA and legitimate domains. Additionally, the false positive rate (0.0824) remains at an acceptable level, which is particularly important for minimizing misclassification of legitimate domains.

Overall, the close alignment between cross-validation and test results confirms that the model maintains consistent performance and does not exhibit significant overfitting.

- 4.) *Error Distribution Analysis*: To gain a deeper understanding of the model's classification behavior, a confusion matrix was generated for the 80,000-sample test set. The confusion matrix analysis shows that the model correctly identified 36,706 benign domains and 34,560 DGA domains, demonstrating strong classification performance. The number of false positives (Type I error) remains relatively low, with only 3,294 benign domains incorrectly classified as DGA, indicating that legitimate user traffic is largely preserved in a production setting. On the other hand, the model produced 5,440 false negatives (Type II error), corresponding to DGA domains that were not detected. These missed instances are likely associated with wordlist-based DGA families that better mimic human-like linguistic patterns, highlighting a key area for future improvement.

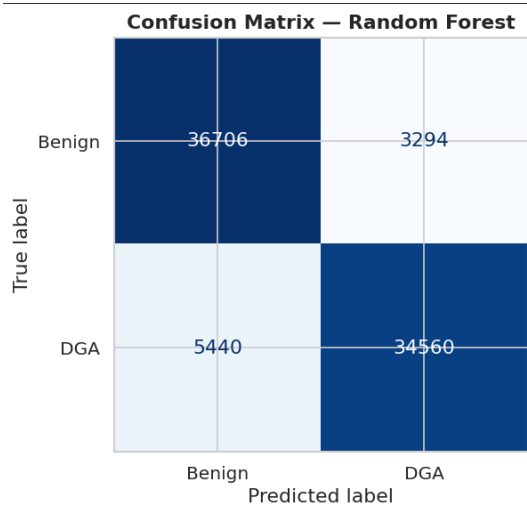


Fig. 3. Confusion Matrix of the Random Forest model on the independent test set

- 5.) *Comparison with Baseline Models:* To contextualize the performance of the proposed Random Forest model, its discriminative capability was compared against a traditional statistical approach, the Shannon Entropy filter, and a theoretical random classifier.

AUC-ROC Analysis: The Receiver Operating Characteristic (ROC) curve illustrates the trade-off between the True Positive Rate (Sensitivity) and the False Positive Rate across all classification thresholds. The Random Forest model achieved a high AUC of **0.9593**, significantly outperforming the Shannon Entropy filter, which yielded an AUC of **0.7053**.

Model Superiority: While Shannon Entropy provides a better-than-random baseline (AUC 0.50), it is insufficient for detecting modern, complex DGA patterns. The large area under the Random Forest curve demonstrates that the integration of 12 structural and linguistic features provides a much more robust detection mechanism compared to simple randomness-based filtering.

Optimal Threshold Performance: The curve for the Random Forest model rises sharply toward the top-left corner, indicating that high recall (detection of DGAs) can be achieved while maintaining a very low false positive rate, which is the ideal behavior for an automated security system.

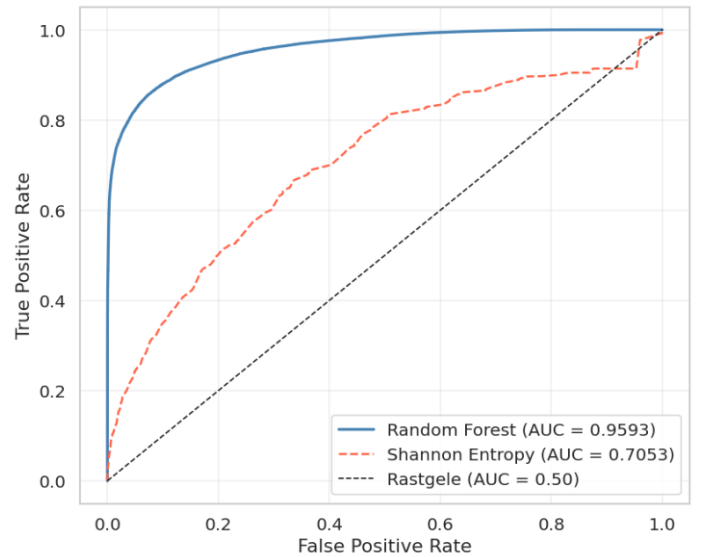


Fig. 4. ROC curve comparison between Random Forest, Shannon Entropy, and Random Chance

- 6.) *Method Comparison Results:* The performance of the proposed Random Forest model was compared against a simple entropy-based baseline and previously published studies. The entropy-based approach performs at near-random levels ($F1 = 0.2163$), highlighting its inability to capture the structural characteristics of DGA domains. In contrast, the Random Forest model achieves a significantly higher F1-score of 0.8878, demonstrating the effectiveness of the extracted feature set. While prior studies such as Tran et al. ($F1 = 0.948$) and Woodbridge et al. ($F1 \approx 0.95+$) report higher scores, the proposed model remains competitive given the dataset diversity and experimental setup.
- 7.) *Cross-Family Generalization Results:* To evaluate generalization क्षमता, the model was trained on 40 DGA families and tested on 10 previously unseen families. In this more challenging cross-family setting, the model achieved an overall accuracy of 0.9242, indicating strong robustness against unseen malware families. However, per-family analysis reveals performance degradation for certain families such as *tinba*, *sisron*, and *ramdo*, where F1-scores remain around 0.32 despite high recall values (>0.91). This suggests that while the model successfully detects a large portion of these domains, it struggles with precision, likely due to similarities between these DGA patterns and benign domains. These findings highlight the limitations of feature-based approaches in capturing more sophisticated,

human-like DGA generation strategies and point to the need for future improvements.

- 8.) *Performance Comparison Between In-Distribution and Cross-Family Settings:* The performance of the Random Forest classifier was evaluated under both in-distribution and cross-family settings to assess its robustness to domain shift. As shown in Table I, the model achieves strong results in both scenarios, with an F1-score of 0.8878 in the in-distribution setting and 0.7888 in the cross-family setting, indicating a moderate decline when generalizing to unseen families. The AUC remains consistently high across both settings (0.9593 vs. 0.9534), suggesting that the model preserves strong discriminative capability despite domain variation. Recall values are also highly comparable (0.8640 for in-distribution and 0.8629 for cross-family), indicating that the model maintains a stable true positive detection rate. In contrast, the false positive rate decreases from 0.0824 to 0.0638 in the cross-family setting, implying a more conservative prediction behavior under distribution shift. Overall, these results indicate that while the model generalizes reasonably well across domains, performance degradation is primarily reflected in the F1-score due to reduced balance between precision and recall. QKWJFHZPLMBXNRST.CC

XI. FINALIZED APPLICATION

The final stage of the project focused on verifying the model's practical utility through a simulated deployment environment within the notebook. This stage confirms that the trained model is ready for integration into real-time systems. In the final stage of the project, the inference capability of the trained model on real-world data was evaluated using a prediction mechanism implemented within the notebook environment.

A. Model Export and Loading Verification

To ensure the model can be used in external applications, it was serialized and saved as `dga_random_forest_model.joblib`. A loading test was performed to confirm that the model's predictive integrity remained intact after being saved. The successful return of consistent results (e.g., [1, 0, 0, 1, 1]) during this loading test verified that the application is finalized and ready for production-level inference.

B. Demo Predictions (In-Notebook Inference)

To simulate the practical usage of the model, the `predict_domain` function was executed on a set of selected domain names. This function processes raw domain inputs through a 12-feature pipeline and generates both the class label and the corresponding DGA probability score.

TABLE I. DEMO PREDICTION TABLE

Domain Name	Class	Prob(P)
google.com	Benign	0.0024
www.amazon.co.uk	Benign	0.3129
bbc.co.uk	Benign	0.3129
qkwjfhzplmbvxnrst.cc	DGA	1.0000
xpqmzlyakwvntf.net	DGA	1.0000
ujfhqpwxmzkatvln.ru	DGA	0.9999

REFERENCES

- [1] D. Plohmann, K. Yakdan, M. Klatt, J. Bader, and E. Gerhards-Padilla, "A Comprehensive Measurement Study of Domain Generating Malware," in Proc. USENIX Security Symposium, 2016, pp. 263–278.
- [2] M. Antonakakis, R. Perdisci, Y. Nadji, N. Vasiloglou, S. Abu-Nimeh, W. Lee, and D. Dagon, "From Throw-Away Traffic to Bots: Detecting the Rise of DGA-Based Malware," in Proc. USENIX Security Symposium, 2012.
- [3] J. Woodbridge, H. S. Anderson, A. Ahuja, and D. Grant, "Predicting Domain Generation Algorithms with Long Short-Term Memory Networks," arXiv:1611.00791, 2016.
- [4] D. Tran, H. Mac, V. Tong, H. A. Tran, and L. T. Nguyen, "A LSTM based Framework for Handling Multiclass Imbalance in DGA Botnet Detection," Neurocomputing, vol. 220, pp. 1–14, 2017.
- [5] M. Zago, M. G. Pérez, and G. M. Pérez, "UMUDGA: A Dataset for Profiling DGA-Based Botnet," Computers & Security, vol. 92, 2020.
- [6] C. E. Shannon, "A Mathematical Theory of Communication," Bell System Technical Journal, vol. 27, no. 3, pp. 379–423, 1948.